

New in Colors · September 2026

Register the workflow. Let a controller keep it converged.

`green.kubernetes` reconciles a custom resource by running the same Green workflow a launcher runs.

`redis-operator` and `doks` are the first two packages on it, verified live on 16 September 2026.

[Download the PDF](#) · [getcolors.ai](#)

A launcher run ends. A controller keeps observing.

One create

Reads `colors.yml`, converges, runs the checks, and exits.

Nobody observes the infrastructure after the exit code.

One controller

Reads a custom resource every poll and observes the infrastructure every `reconcileInterval`.

Converges only when the observation does not match the desired state.

The controller is an optional Green library. It calls registered package functions and loads no code from a resource.

The controller in one slide

- The loop polls the custom resources with kubectl and coalesces queued work per resource.
- One process-wide lock per infrastructure state identity serializes whole workflows, across types and controller instances.
- The controller adds the finalizer `colors.getcolors.ai/infrastructure` before any effect.
- Every status write carries the `resourceVersion` of the snapshot it came from.

`spec.state` running · `spec.suspend` · `spec.reconcileInterval` 60s · `spec.deletionPolicy` Retain or Destroy · the `colors.getcolors.ai/reconcile-request` annotation

A package registers five callbacks

Callback	Returns
<code>validate</code>	Nothing for valid configuration, or a collection of errors
<code>identity</code>	A stable value naming the backend and the profile
<code>observe</code>	<code>exists?</code> , <code>matches?</code> , and <code>ready?</code> from a read-only look at the infrastructure
<code>converge</code>	The outcome of the package's converge workflow
<code>delete</code>	The outcome of the package's delete workflow, repeatable after partial cleanup

`converge` and `delete` call the same `green.workflow` graphs a launcher runs.

One controller process. It implements no distributed leases or fencing.

redis-operator provisions one Droplet per RedisDeployment

The adapter

Imports the pinned `redis` package workflow.

Only a confirmed provider 404 counts as absence. An authentication, provider, or state-read error is a retryable failure, never permission to recreate.

The guards

`compute-prevent-destroy` stays on while the controller converges.

Only an explicit `deletionPolicy: Destroy` lifts it, under the same identity lock.

The controller image and its volume

On the volume

A failed converge or delete is kept at `/data/work/<profile>/failures/`, 20 newest, every `COLORS_PAR_*` value masked to `***`.

SSH keys, the convergence record, and the dependency caches share the same volume, so a restart resolves nothing.

The image

`scripts/image.sh` builds `linux/amd64` from the checked-out commit and pushes it to the deployment's registry.

The deployment pins the printed digest as `image` in `colors.yml`.

doks is the cluster underneath

One managed cluster

DigitalOcean DOKS or Vultr VKE, named after the profile, through colors-compute's `managed-kubernetes` kind.

State lives under

`<profile>/compute/managed-kubernetes.tfstate`.

One optional registry

Deployment-owned, named after the profile, integrated with the cluster.

DOKS then places the pull Secret in every namespace.

```
./green build | create | check | kubeconfig | registry | delete
```

One live cycle, 16 September 2026

Step	Result
<code>doks</code> create	Cluster and registry in 6 min 16 s. A second create found nothing to do in 13 s.
First operator create	Ready at generation 1
Image update	The controller rolled and ran no redundant create
Drill	Deleted Droplet 600954837; replacement 600968621 healthy after 5 min 21 s, resource UID unchanged
Rehearsal	Suspended at generation 2, resumed at generation 3
Restart	Passed on the second attempt; the new pod was up in about 20 s from cached dependencies
Destroy delete	The finalizer destroyed the Droplet, then the verb removed the namespace and the CRD
Shutdown	Cluster and registry destroyed. Account empty.

Four fixes came out of the live run

- `check` read `phase=Reconciling reason=Reconciling` on a healthy resource. It now polls every 5 s for up to 180 s through the periodic pass.
- A failed first Ansible attempt left nothing readable. The controller now retains masked failure logs, and `check` prints `failures retained: N`.
- A probe exec hit `Unexpected end of ZLIB input stream` in a pod still unpacking its toolchain. Every exec now waits for that pod's own `RedisDeployment controller running` line, and the caches live on the volume.
- The delete patch failed with `the server rejected our request due to an error in our request` because status writes had moved the `resourceVersion`. Patches now retry when only status moved.

One item stays open

On 3 of 4 creates over two days, the first Ansible attempt on a fresh Droplet failed:

```
converge outcome=failed step=:redis/ansible exit=2
```

The controller's retry converged about two minutes later with nothing changed. The cause is unknown.

The first build discarded the play output by design, and the failure did not recur once retention existed. The next occurrence is readable with `kubectl exec` and `cat` `/data/work/<profile>/failures/<file>`.

A gate that fails a create on the first Failed pass is wrong. The create wait tolerates Failed because the controller retries.

Find them in the catalog

Package recipes now carry a `shape`, one of single-node, multi-node, kubernetes, operator, or local. It renders as a chip on the card and a filter on `/skills`. Source pages list `artifacts`: the CRD, the controller image, the rendered manifests. A Context Skill inherits the shape of its companion package.

```
npx skills add getcolors/doks --skill package-doks-green  
npx skills add getcolors/redis-operator --skill package-redis-operator-green  
npx skills use getcolors/skills --skill redis-operator-kubernetes
```

[getcolors.ai/getcolors/doks](#) · [getcolors.ai/getcolors/redis-operator](#) · [redis-operator-kubernetes](#)

Online · Free · One hour

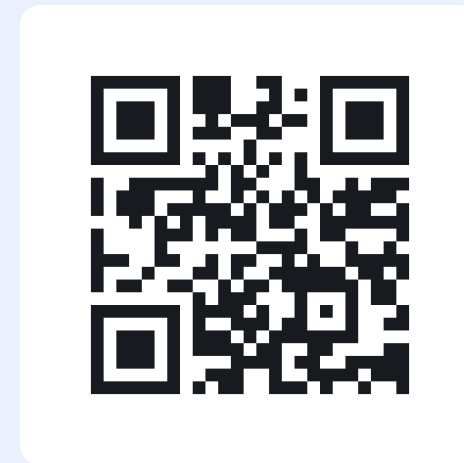
Join the Community Town Hall

Friday, 18 September 2026

17:00 CEST / 15:00 UTC

See a live provisioning demo, discuss the roadmap, and bring your infrastructure questions.

Register for the Town Hall



luma.com/ci9bek4c